Methodology: Multi-Day SOC-Aware EV Charging Simulation

`Distribution-Grid-EV-CA/multiday_charging/`

Replication documentation

June 4, 2026

Abstract

This document describes the `multiday_charging` subproject: a stand-alone Python simulation that samples EV charging sessions over many days using battery state-of-charge (SOC) logic, configurable charging-frequency preferences, and an empirical session pool. It complements the main grid pipeline documented in `pipeline_methodology.tex`, which builds charging *events* from CSTDM trips (SDPTM, LDPTM, ETM) and only attaches empirical start times and power in stage 05.

Contents

1 Purpose and relation to the main pipeline

The main pipeline (`01_01`–`07_05`) answers: *where and how many* charging events occur on feeders over adoption years, then copies *when and how fast* from real charger logs. The `multiday_charging` subproject answers a different question: *given a fleet with heterogeneous batteries and driving, when do vehicles choose to charge, and what hourly load shapes emerge over 365 days?*

Aspect	Main pipeline (03–05)	multiday_charging
Trip source	CSTDM SD/LD/ETM tables	Synthetic daily VMT + optional SDPTM calibration
Charge timing	Per trip / per corridor stop	SOC + preferred interval (1–7 days)
Empirical data	Stage 05 only (`05_02`)	Required pool at build time
Spatial output	Block / feeder	Fleet-aggregate hourly (optional zone column)

2 Software layout

```
multiday_charging/
  config.py # MultiDayConfig dataclass (all parameters)
  fleet.py # Battery mix, access flags, interval preference
  trips.py # Daily VMT + work/public availability
  session_pool.py # Load charging_session_all_clean.pkl
  choice.py # Location / DC-L2 / housing weights
  sampling.py # SOC-aware multi-day sampler
  load_profile.py # Hourly matrices and percentile bands
  run_multiday_charging.ipynb
  outputs/ # Plots and arrays (created at run time)
```

Parent helpers: `../common.py` (`read_rds_like`, `save_rds_like`). Input builder: `../build_multiday_inpu`

3 Required inputs

3.1 Empirical session pool (required)

Path: `data/charging data/charging_session_all_clean.pkl`

Built by `build_multiday_inputs.py`:

- **Home** — PEV-Profiles L1/L2 (`PEV-Profiles-L1.xlsx`, `PEV-Profiles-L2.xlsx`): 10-minute power profiles converted to sessions.
- **Work / public** — `intermediate_geolabeled_with_density_category 2.csv`: non-residential sessions (property-type mapping follows `Reference_Code/6_charging_behavior_synthetic_l`

Columns used: `charge_type` (`home`, `work`, `public`), `charger_level` (`L2`, `DC`), `housing` (home only), `energy` (kWh), `power` (kW), `start_hour`, `end_hour`. Bins 1–8 match stage 05: $(0, 5], \ldots, (80, \infty]$ kWh.

3.2 Optional SDPTM calibration

Path: `data/mobility_data/CSTDM_processed/EV trips_new_SDPTM_sample42.pkl`

When `CALIBRATE_FROM_REAL_TRIPS=True` in the notebook:

- Per-vehicle `mean_daily_vmt` $\leftarrow \sum$ `Dist` by `SerialNo`.
- `work_access` $\leftarrow$ configured share $\wedge$ vehicle ever has `ChargeType=W`.

Day-to-day VMT noise still uses `vmt_day_to_day_cv`; consider filtering trips to one `year` if multi-year inflation is unwanted.

4 Configuration (`MultiDayConfig`)

All parameters are set in one notebook cell (defaults in `config.py`).

4.1 Fleet and energy

- `battery_mix` — {kWh: share} capacity distribution.
- `efficiency_mi_per_kwh` = 3.0 — matches parent rule demand = `dist`/3.
- `reserve_soc` — minimum pack fraction retained; usable energy = `battery_kwh`(1 − `reserve_soc`).

4.2 Charging-frequency matrix

`charge_interval_probs`: map $\{1, \ldots, 7\} \to$ probability. Each vehicle draws `interval_pref_days`. Charging may occur earlier if the pack would deplete.

4.3 Charging-location matrix

`location_weights` default aligns with `Reference_Code` `RAW_SHARES`:

Label	Weight
Home (`Residential_L1_L2`)	0.68
Work (`Workplace_L2`)	0.04
Public (`Public_L2` + `DCFC`)	0.28

Feasibility: `home_access_share`, `work_access_share`, daily `work_trip_prob`, `public_trip_prob`. Infeasible locations get zero weight; remaining weights are renormalised (public is the backstop).

4.4 Sub-type weights (within location)

- `public_level_weights` — DC vs L2 when sampling public (default 0.20 / 0.08, normalised).
- `home_housing_weights` — single vs multi family (56.4 / 38.9, same as `05_02`).

4.5 Daily driving and variability parameters

Mean VMT: `mean_daily_vmt` (default 32–35 mi) is a planning prior, close to `AVG_DAILY_VMT_PER_EV = 35` in `Reference_Code/6_charging_behavior_synthetic_load.ipynb`. With SDPTM calibration, each vehicle's personal mean is overwritten by $\sum$`Dist` per `SerialNo` (optionally filter to one `year` to avoid multi-year inflation).

Coefficient of variation (CV): `vmt_between_vehicle_cv` and `vmt_day_to_day_cv` are *not* probabilities in $[0, 1]$. They are dimensionless spreads $\mathrm{CV} = \sigma/\mu$:

- `vmt_between_vehicle_cv` (e.g. 0.45) — spread of `mean_daily_vmt` across vehicles via a lognormal draw in `fleet.py`.
- `vmt_day_to_day_cv` (e.g. 0.55) — day-to-day multiplier $\varepsilon_{v,d}$ around each vehicle's mean in `trips.py`, with $\mathrm{median}(\varepsilon) = 1$.

Implementation: $\sigma = \sqrt{\log(1 + \mathrm{CV}^2)}$ for the underlying lognormal so the target CV is preserved.

Other: `no_travel_prob` (e.g. 0.08) is a true probability — fraction of days with zero miles.

4.6 Parameter provenance (quick reference)

Parameter	Typical source
`public_level_weights` 8919 / 29229	`05_02` public charger counts (DC vs L2)
`home_housing_weights` 56.4 / 38.9	`05_02` housing-unit weights
`location_weights` 0.68 / 0.04 / 0.28	`RAW_SHARES` in reference notebook
`home_access_share`, `work_trip_prob`, ...	Engineering defaults for feasibility
`charge_interval_probs`	User-tunable; not from CSTDM

5 Simulation algorithm

5.1 Step 1: Build fleet (`fleet.py`)

Draw per vehicle: `battery_kwh`, `interval_pref_days`, `home_access`, `work_access`, `mean_daily_vmt`.

5.2 Step 2: Daily inputs (`trips.py`)

For each (vehicle, day):

$$\texttt{vmt}_{v,d} = \texttt{mean\_daily\_vmt}_v \times \varepsilon_{v,d}, \quad \varepsilon \sim \text{Lognormal},\ \text{median}(\varepsilon) = 1 \tag{1}$$

Zero out VMT with probability `no_travel_prob`. Set `work_available` and `public_available` from Bernoulli draws and access flags.

5.3 Step 3: Session pool (`session_pool.py`)

Load empirical pool; assign `bin` from `energy`; assign `eventID`.

5.4 Step 4: Multi-day sampling (`sampling.py`)

For each day d and vehicle v:

1. Energy consumed: $e_{v,d} = \texttt{vmt}_{v,d}/\texttt{efficiency\_mi\_per\_kwh}$.
2. Accumulate E_v^{cum} += $e_{v,d}$; increment `days_since`.
3. **Charge if** $E_v^{\text{cum}} \geq \texttt{usable\_kwh}_v$ **or** $\texttt{days\_since} \geq \texttt{interval\_pref\_days}_v$ (and VMT > 0 that day).
4. Feasible locations: home if `home_access`; work if `work_available`; public if `public_available`.
5. Draw location from renormalised `location_weights`; draw housing or DC/L2 sub-type.
6. Compute recharge **demand** $= \min(E_v^{\text{cum}}, \texttt{usable\_kwh}_v)$; map demand to energy bin b (same cutpoints as stage 05).
7. Sample one empirical row with matching (`charge_type`, sub-type, b); copy `start_hour`, `end_hour`, `power`, and pool `energy` as `energy_session`.
8. Reset E_v^{cum} and `days_since` (full recharge assumption).

5.4.1 Matching demand kWh to empirical session kWh

The simulation stores two energy fields per sampled session:

- `energy_demand` — kWh the vehicle needed since the last charge (from accumulated driving, capped by usable pack).
- `energy_session` — kWh from the *drawn* historical session in the pool.

They are **not forced to be equal**. Matching is **stratified by bin only**:

1. Driving sets `energy_demand` (e.g. 9.3 kWh).

2. Demand maps to bin b (e.g. bin 2: $(5, 10]$ kWh).
3. A random pool row is drawn with the same `charge_type`, housing or DC/L2 sub-type, and bin b (e.g. 8.2 kWh).

This mirrors stage `05_02`: synthetic events get `demand` = `dist`/3 and a bin; empirical `eventID` supplies timing and power without rescaling session energy to synthetic demand.

What drives hourly load plots: `load_profile.py` uses empirical `power` (kW) and `start_hour`/`end_hour` only — not `energy_demand`. Integrated energy over active hours tracks `energy_session`, which is only loosely tied to demand through the shared bin. The design is bootstrap / stratified resampling: SOC logic sets *when* and *how large* (bin); the pool sets *time-of-day shape* and charger power.

5.5 Step 5: Load profiles (`load_profile.py`)

Build $[n_{\text{days}}, 24]$ matrix: constant power over session hours (wrap at midnight). Outputs: daily curves, 5th–95th hourly percentiles, optional split by `charge_type`.

6 Outputs

Default directory: `multiday_charging/outputs/`

- `daily_load_365_lines.png`
- `hourly_load_percentile_band.png`
- `mean_load_by_type.png`
- `daily_hourly_load_kW.npy`
- `hourly_percentile_band.csv`
- `sampled_sessions.parquet` (if supported)

7 How the main pipeline samples SD, LD, and ETM charging

This section documents stages 03 and 05 of the parent pipeline. `multiday_charging` does not run these steps; it is included so both documents stay self-contained.

7.1 Short distance (SDPTM) — `03_01`

Input: `EV trips_new_SDPTM_sample42` with `ChargeType` $\in \{H, W, P\}$ from trip purpose.

Logic:

1. Group trips into tours (`SerialNo`, `Person`, `Tour`).
2. For each tour, list which of $\{H, W, P\}$ appear among trips.
3. Form seven valid location *patterns* (e.g. home-only, home+work, all three) with EVMT survey shares (53% home-only, 8% work-only, ...).
4. Normalise shares to the tour; draw one pattern (seed 12).
5. A trip creates a charging event if its `ChargeType` matches an active location in the drawn pattern.
6. Sum `Dist` over trips assigned to each event → `chargeDist`; attach TAZ `J`, `Time`, `year`.

Output: `charge_event_new_SDPTM_sample42_draw12`. No empirical session times yet — only *that* charging happens, *where* (H/W/P), and *how far* (miles).

7.2 Long distance (LDPTM) and external (ETM) — `03_02`

Reference: `R_Yanning/03_02_charging events_LD_ETM.R`. Options 1–3 are explored in R but **not saved**; both the R workflow and the Python port persist **Option 4 only** to `charge_event_new_LDPTM_samp` and `charge_event_new_ETM_sample42`.

Inputs:

- EV trips (`EV trips_new_LDPTM_sample42` or `EV trips_new_EXT_sample42`).
- Parsed skim: `parsed_100000_LDPTM` or `parsed_100000_ETM` — one row per TAZ on the shortest-path sequence for each (I, J).

7.2.1 What `TAZfreq` means

On the *full* skim table (all modeled long-distance paths, before joining EV trips), Yanning defines:

```
dist.ld[, TAZfreq := .N, by = TAZway] # R data.table
```

`TAZfreq` for TAZ k = number of skim **rows** where `TAZway =` k. Because each origin–destination pair has many rows (one per TAZ along the path), a high `TAZfreq` means that TAZ appears on many $I \to J$ routes in the LD (or ETM) network — a proxy for a **major corridor / hub**, not census population, charger inventory, or EV trip counts.

What it is not: EV trip frequency, distance since last charge, or geographic area.

7.2.2 Option 4: which TAZ gets a charging event

Option 4 does **not** place a charger every 100 miles along the odometer. It selects TAZs **on the trip's path** that are also **high-`TAZfreq`** relative to other TAZs on that same path, targeting about $\lceil D/100 \rceil$ stops.

Step-by-step (per trip).

1. **Target stop count:** $n_{\text{charge}} = \lceil D/100 \rceil$ (e.g. 220 mi $\Rightarrow$ 3 stops).
2. **Path length:** $n_{\text{TAZ,trip}}$ = number of skim rows for this `TripID` on (I, J).
3. **Quantile probability:**
$$q_{\text{cut}} = 1 - \frac{n_{\text{charge}}}{n_{\text{TAZ,trip}}} \tag{2}$$
Many path TAZs but few required stops $\Rightarrow$ high q_{cut} $\Rightarrow$ stricter filter.
4. **Cutoff on this path only:** Let $F = \{$`TAZfreq of TAZs on this trip's path`$\}$. Then
$$\texttt{freqCUT} = Q_{q_{\text{cut}}}(F) \tag{3}$$
(R: `quantile(TAZfreq, probs=qtCUT, by=TripID)`; Python: `np.quantile` per trip.)
5. **Keep TAZs:**
   - LDPTM: `TAZfreq > freqCUT`
   - ETM: `TAZfreq >= freqCUT` (R line 160; slightly looser than LD)
6. **Attributed miles per stop:** `chargeDist` $= D/n_{\text{TAZ,charge}}$ where $n_{\text{TAZ,charge}}$ is the count of kept TAZs. Distance is split **evenly** across selected stops, not by actual spacing along the path.

LD labeling after TAZ selection.

- Corridor TAZs (`TAZway`$\neq J$): `ChargeEventType = P` (public), `other = corridor`.
- Destination (`TAZway = J`): `ChargeEventType` = trip `ChargeType` (H/W/P), `other = destination`.

ETM. Same Option 4 on `parsed_100000_ETM`; trips are inbound to California; output keeps `ChargeType` (typically public). No `ChargeEventType`/`other` corridor split in the saved ETM file.

Numeric example. Trip 220 mi, 40 TAZ rows on path: $n_{\text{charge}} = 3$, $q_{\text{cut}} = 1 - 3/40 = 0.925$. If `TAZfreq` on those 40 TAZs ranges 50–5000, `freqCUT` might be ≈ 4200. Kept TAZs have `TAZfreq`> 4200 — often $\sim$3 busy highway TAZs, not necessarily spaced every 73 mi. Each gets `chargeDist` $\approx 220/3 \approx 73$ mi.

What Option 4 is not.

- Not all TAZs on the path (Option 1).
- Not a fixed global top-10% of TAZs (Option 3 uses 90% quantile fixed).
- Not geometric "every 100 miles" along the route polyline.

7.2.3 Consistency with Yanning's R script

The Python port `03_02_charging_events_LD_ETM.py` matches the **saved** R Option 4 logic:

Step	R (`03_02_charging events_LD_ETM.R`)
`TAZfreq`	`.N, by=TAZway` on full `dist.ld` / `dist.etm`
n_{charge}	`ceiling(Dist/100)`
q_{cut}	`1 - nCharge/nTAZtrip`
`freqCUT`	`quantile(TAZfreq, probs=qtCUT, by=TripID)`
LD filter	`TAZfreq > freqCUT` (line 88)
ETM filter	`TAZfreq >= freqCUT` (line 160)
`chargeDist`	`Dist/nTAZcharge`

Minor implementation notes: R `quantile` vs NumPy may differ slightly at bin edges; ETM trip–path join in R is trip-driven (`allow.cartesian=TRUE`) whereas Python uses a left merge on `dist` — results align when every ETM trip has a skim match.

7.3 Where empirical sessions attach — `05_02`

Stages 03–04 produce **synthetic events**: which TAZ or block, how many events, `chargeDist` or `dist` (miles). Energy need for binning:

$$\texttt{demand (kWh)} = \frac{\texttt{dist}}{3} \tag{4}$$

Eight bins $(0,5], \ldots, (80,\infty]$ on both synthetic demand and pool `energy`.

Stage `05_02` then:

1. Assigns public DC vs L2 (charger counts 8919 / 29229) and home housing (56.4 / 38.9); LD corridor trips often forced to DC.
2. For each stratum (type, housing or level, bin), draws `eventID` from `charging_session_all_clean` (seed 36).
3. Copies `start_hour`, `end_hour`, `power`, `energy` — same stratified bootstrap principle as `multiday_charging`, but spatially at feeder/block level after stage 04.

Stage 06 expands each event to hourly feeder load using constant `power` over [`start_hour`, `end_hour`].

8 Run commands

```
# Build inputs (once)
python build_multiday_inputs.py

# Interactive run
jupyter notebook multiday_charging/run_multiday_charging.ipynb
```